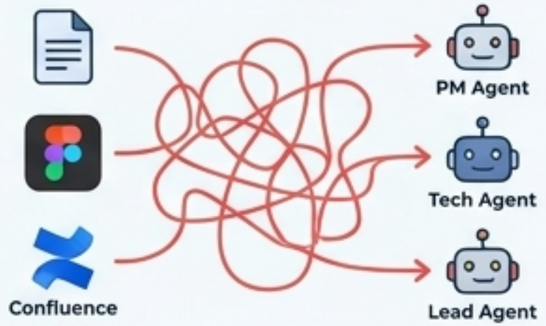
部門共用 AI 助理技術提案

以 Claude Code 為核心, 打造具備「中央快取」與「多視角分發」的營運中樞

現況挑戰：資源浪費與資訊孤島



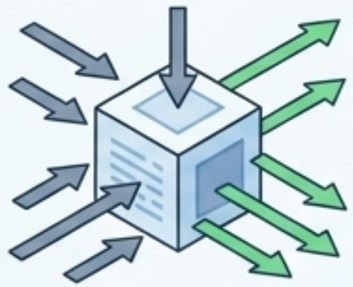
N倍 Token 消耗, 資訊不同步, 語境破碎

重複消耗 Token, 跨平台語境斷層, 缺乏追蹤

重工與浪費: 相同長文本被重複讀取, API 成本飆升

缺乏全局視角: 資料散落, 個體無法掌握進度

核心目標：建立部門語境供應商

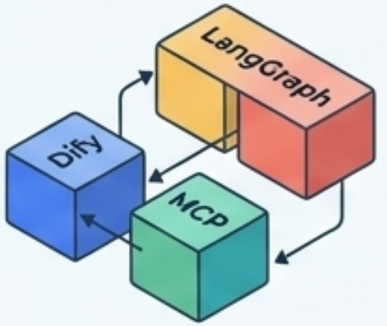


實現「**一次輸入，多路輸出**」

為部門建立**統一**的**中央資料快取**

解決**資訊孤島**與**資源重複消耗**問題

最終建議：Dify + LangGraph + MCP 混合架構



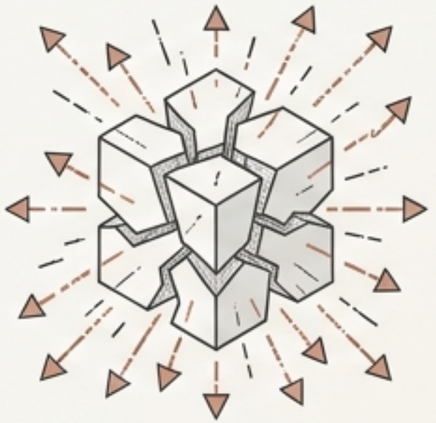
結合**數據管理**、**核心編排**與**通訊協議**

無縫對接 **Claude Code 終端**

提供**高效**、**統一**的 **AI 營運中樞**

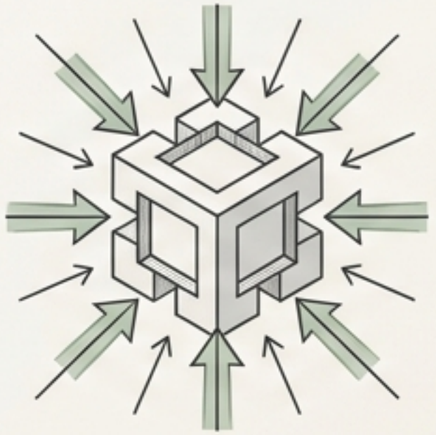
NotebookLM

執行摘要



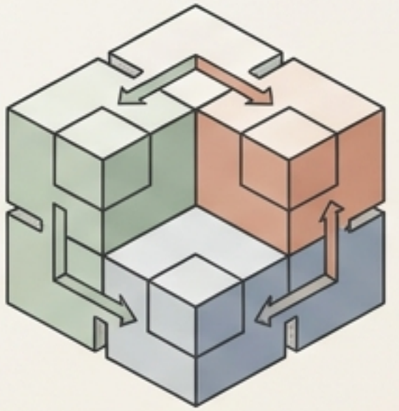
現況挑戰

資源浪費與資訊孤島。重複消耗 Token 爬取同份資料，跨平台（Figma, Confluence）語境斷層會議 Action Items 缺乏追蹤。



核心目標

建立部門語境供應商。實現「一次輸入，多路輸出」，為部門建立統一的中央資料快取。

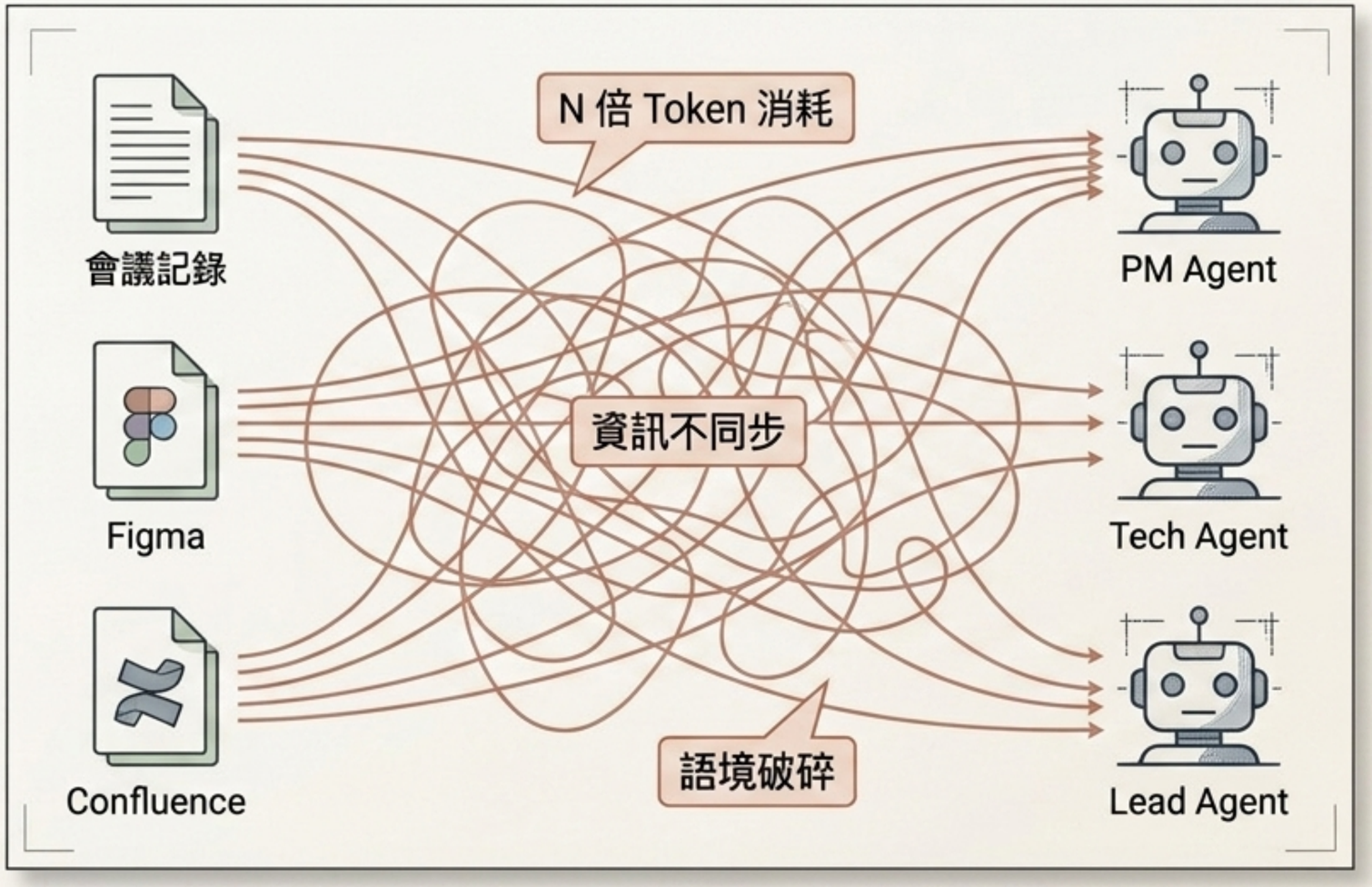


最終建議

Dify + LangGraph + MCP 混合架構。結合數據管理、核心編排與通訊協議，無縫對接 Claude Code 終端。

NotebookLM

當前痛點：資源浪費與資訊孤島



重工與浪費

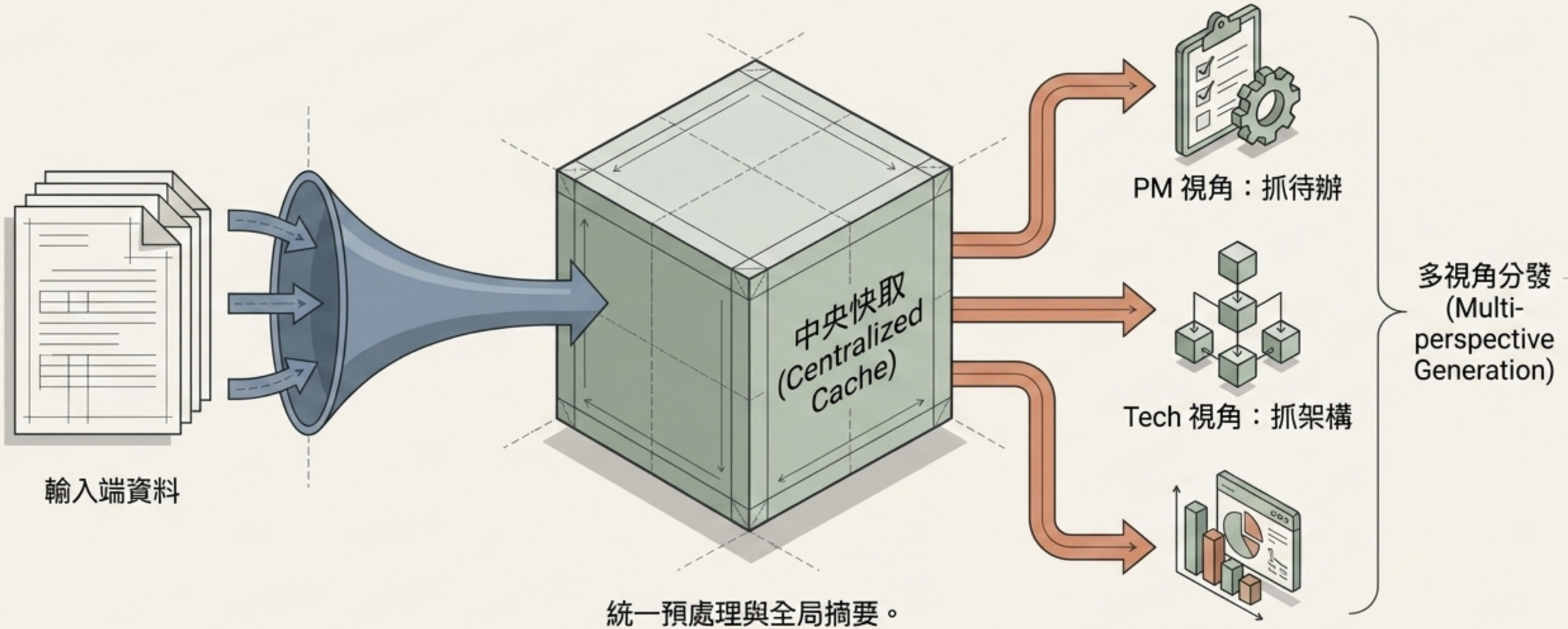
相同長文本（如會議逐字稿）被多人各自的 Agent 重複讀取，導致 API 成本無意義飆升。

缺乏全局視角

設計與開發資料散落各處，個人獨立 Agent 無法掌握專案全局進度，極易產生幻覺或遺漏。

NotebookLM

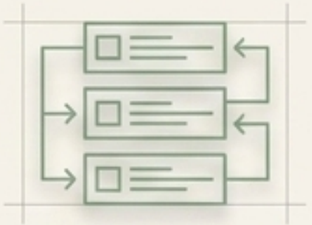
架構願景：一次輸入，多路分發



統一預處理與全局摘要。
效益：僅消耗一次長文本 Token。

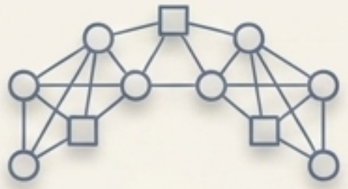
NotebookLM

四大核心支援場景



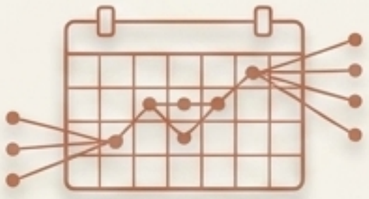
中央知識庫管理

自動去重並解析會議逐字稿，透過高效率 RAG 管道存入共用資料庫。



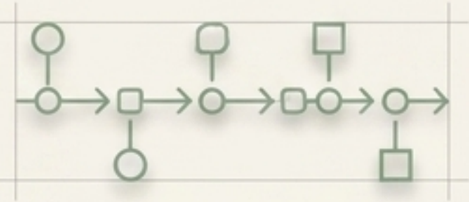
跨平台語境整合

支援 MCP (Model Context Protocol) ，自動從 Figma 抓取異動、Confluence 提取需求。



部門排程與籌備

比對多人空檔 (MSGraph) ，透過多代理協作生成符合 SOP 的會議議程。

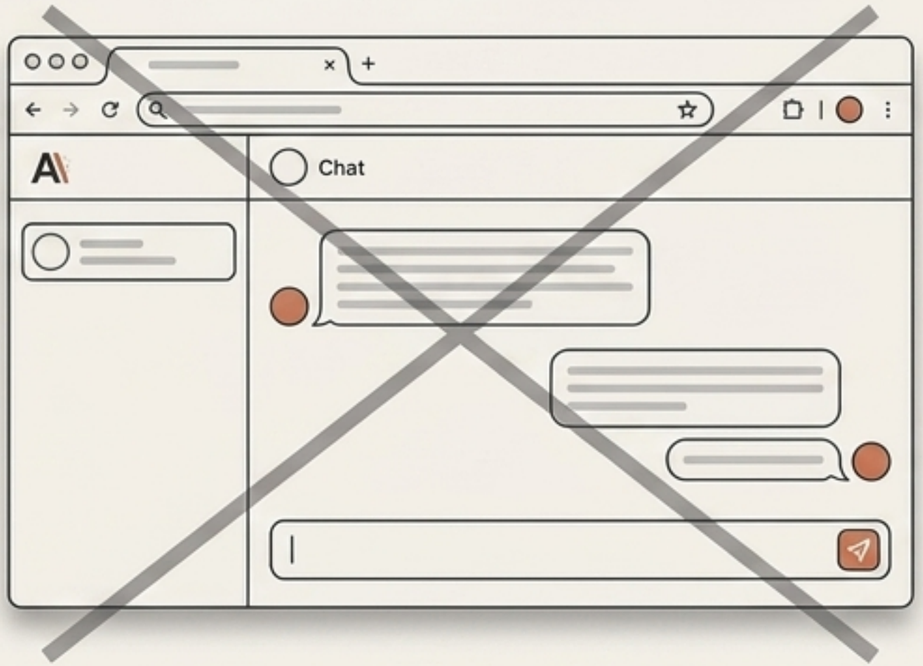


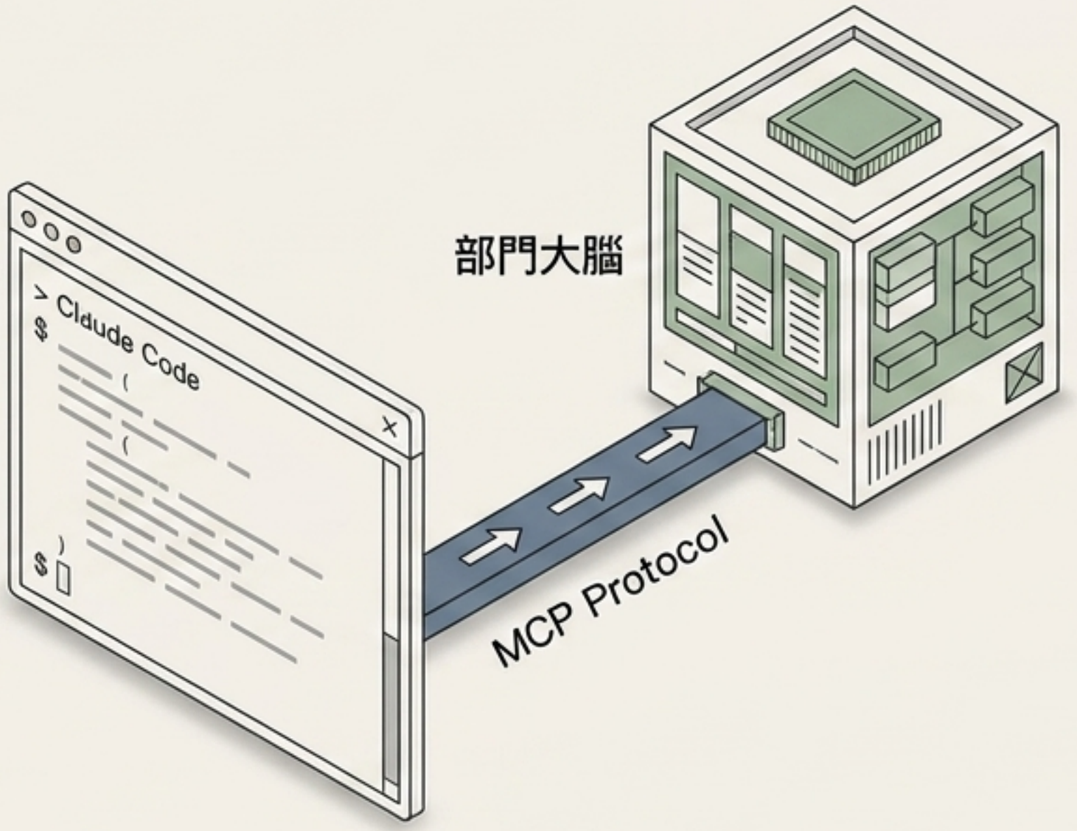
長期工作追蹤

運用 Checkpointers 追蹤任務狀態（未開始/進行中/已完成），支援狀態回溯與歷史記憶。

NotebookLM

互動典範轉移：助理即工具





無縫終端體驗

助理封裝為 MCP Server，直接成為開發者終端機內的超級工具，無需切換視窗。

Agent-to-Agent 協作

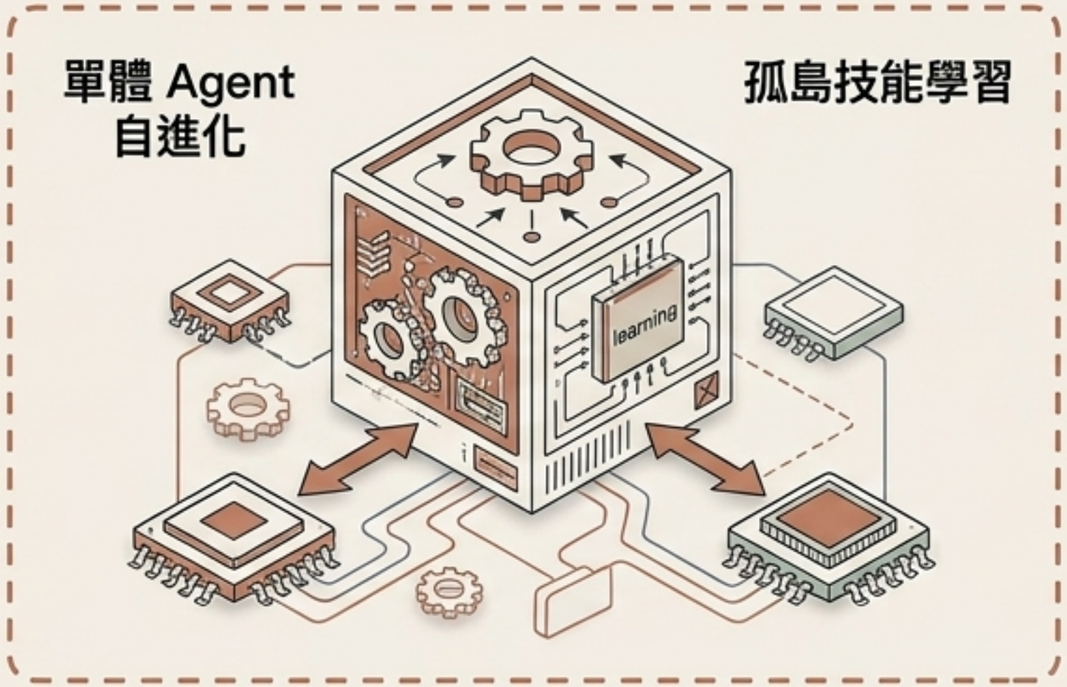
使用者在 Claude Code (CC) 的個人 Agent 透過 MCP 自動調用部門助理，獲取最新語境。

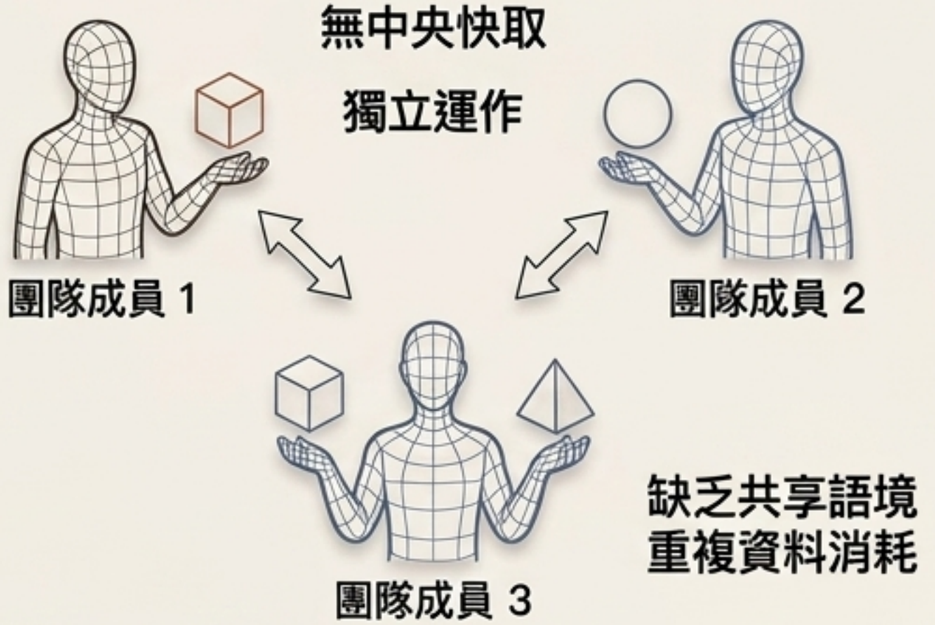
消滅幻覺

確保所有團隊成員的個人 Agent 都在一致的部門規範與即時進度下思考。

NotebookLM

為什麼不用 Hermes？（單體 Agent 框架的侷限）





Hermes 的強項

專注於「個體自進化」與單一 Agent 的新技能學習。

不適配本專案原因

1. 無法建立中央快取：會演變成多個聰明但各自獨立的助理。
2. 未能解決核心痛點：無法避免多人重複讀取資料、重複消耗 Token 的架構問題。

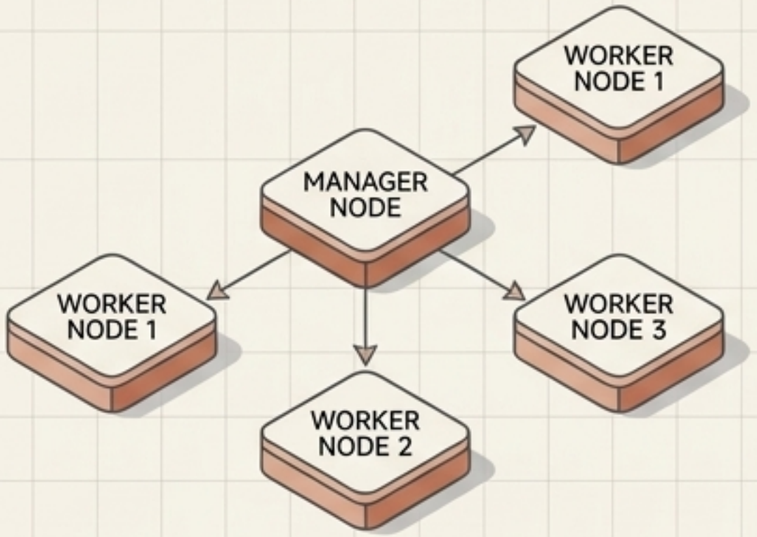
結論：本部門需要的是「編排框架 (Workflow Engine)」，而非單體 Agent。

NotebookLM

工作流引擎切入點：LangGraph vs. CrewAI

Workflow Engine 的絕對優勢：利用圖形結構實現「一次輸入 ➡️ 多路並行處理」。

CrewAI

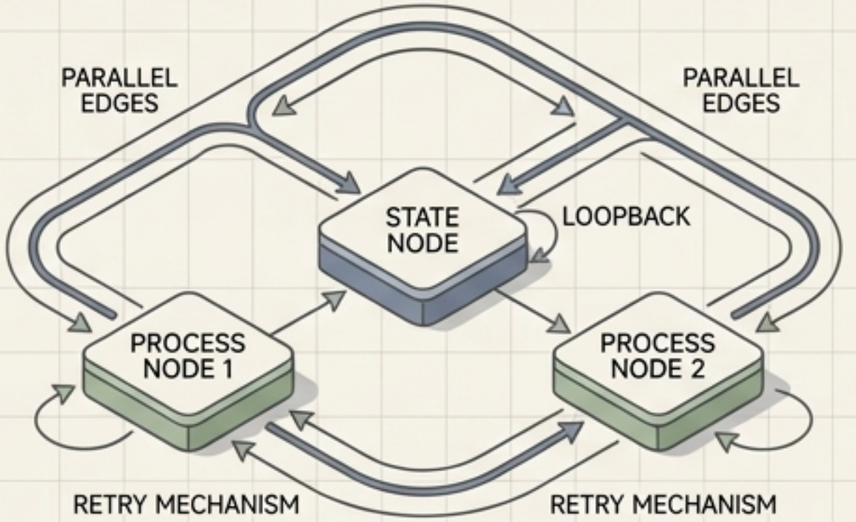


定位：團隊感、快速原型。

特性：透過直覺的 YAML 或 Python 宣告 Role/Backstory，開發快速，適合創意籌備。

侷限：狀態管理較黑盒，長任務易浪費 Token。

LangGraph



定位：成本極致優化、精準控制。

特性：具備狀態持久化 (State Memory) 與 Map-Reduce 架構，可精準控制邊緣 (Parallel Edges) 與單獨重試機制。

NotebookLM

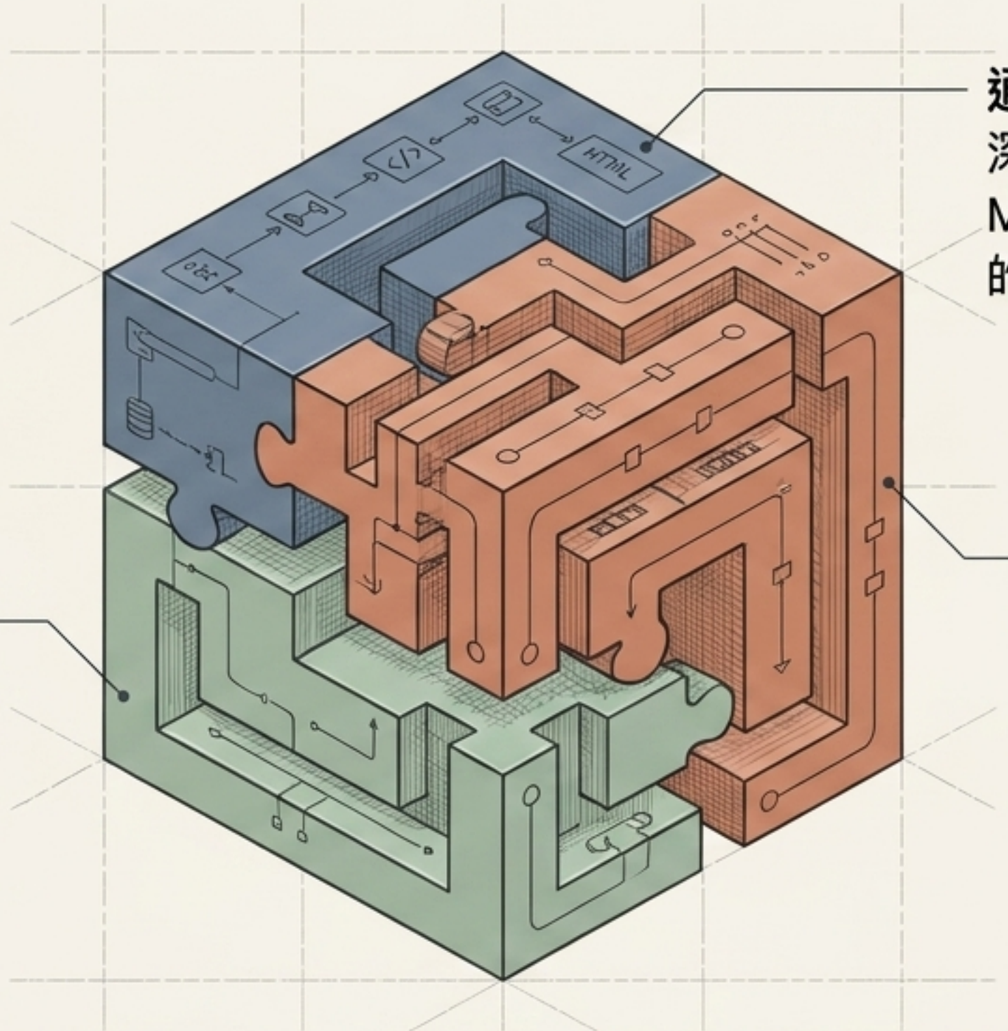
技術選型評估矩陣

維度	CrewAI	LangGraph	Dify
開發速度與交付	High	Low	High (拖拉介面)
狀態持久化與長效記憶	Medium	High (Checkpointers)	Medium
複雜邏輯與條件分支	Medium	High (有環狀態機)	Medium
RAG/知識庫管理門檻	Low	Low	High (內建切片向量化)
MCP 支援成熟度	Low	High	High

LangGraph 在「狀態持久化」具絕對優勢；Dify 在「低代碼 RAG」表現最佳。

NotebookLM

綜合建議：不要單選，要完美融合



通訊接口層（MCP）：
深度整合。將工作流封裝為 MCP Server，成為 Claude Code 的超級工具。

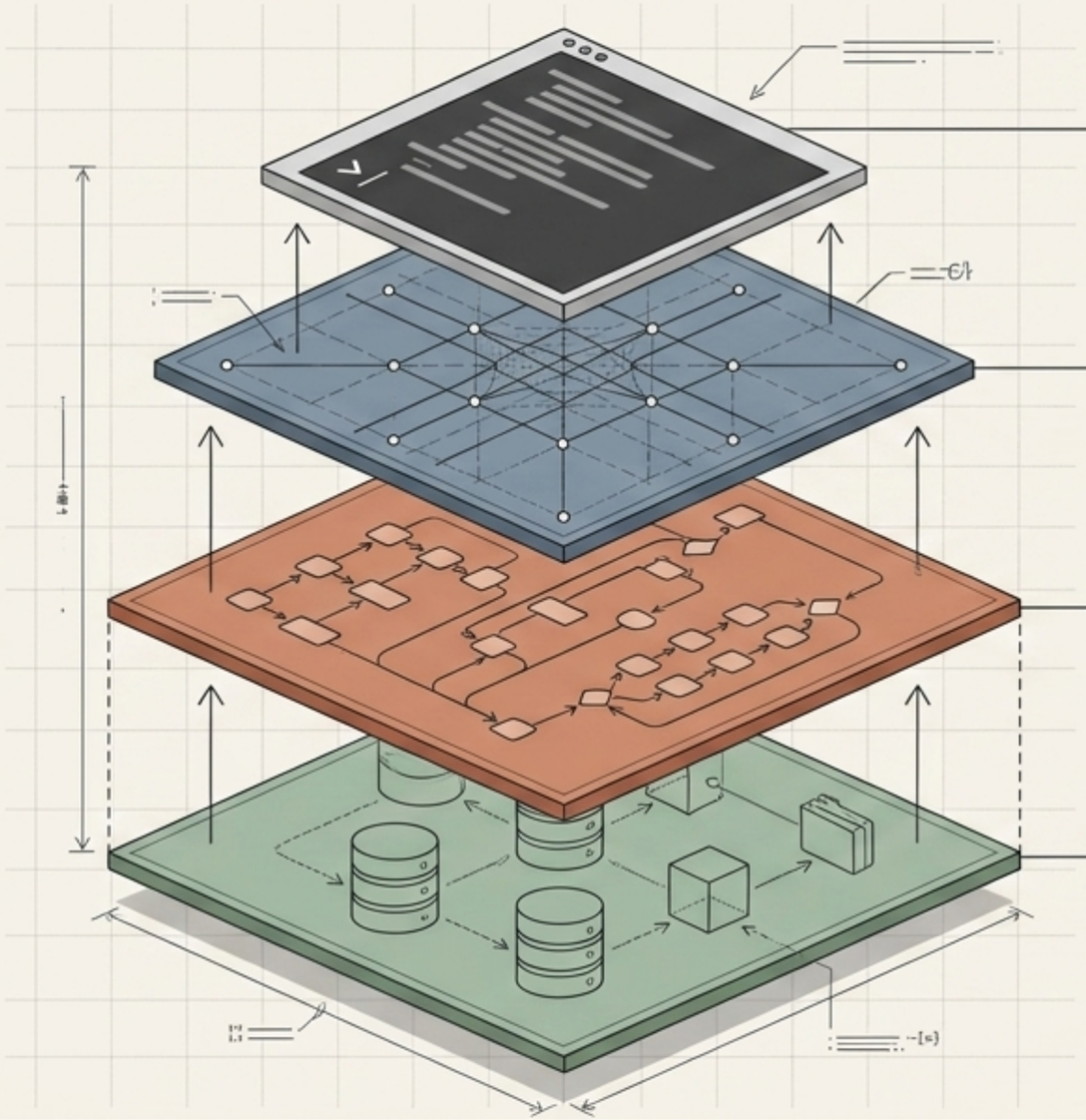
核心編排層（LangGraph）：
精確控制。處理排程衝突、多路並發（Map-Reduce）與長效任務狀態追蹤。

數據工程層（Dify）：
敏捷交付。提供視覺化低代碼介面，輕鬆管理 RAG 知識庫與數據接入。

核心架構：以 Dify 為入口，LangGraph 為核心邏輯的 MCP 驅動架構。

NotebookLM

系統架構藍圖



應用層 Client - Claude Code

使用者終端 (Agent-to-Agent 自動調用)。

通訊層 Protocol - MCP

MCP Server (封裝 API)。

中層 Logic Layer - LangGraph

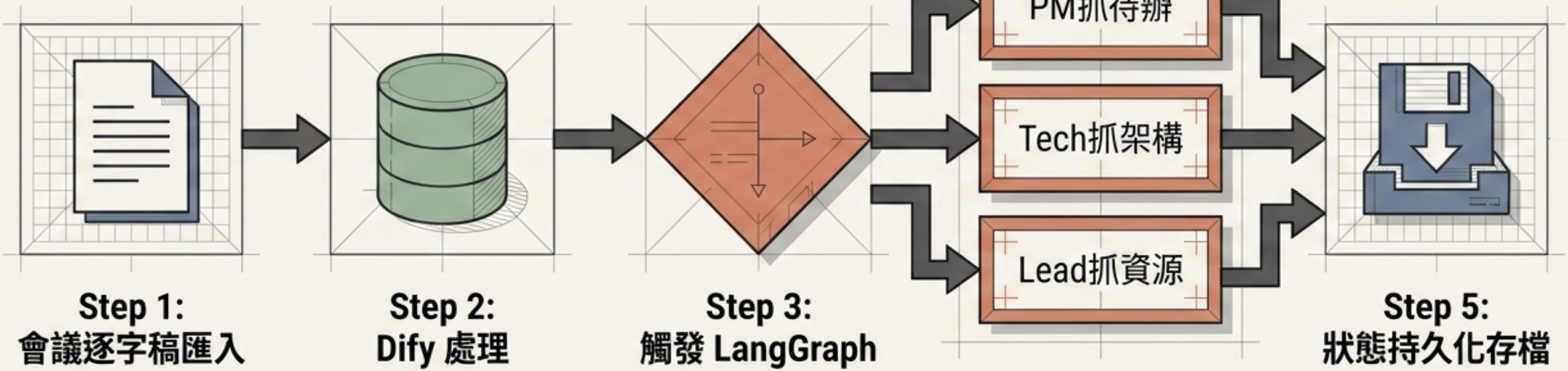
任務狀態機，衝突排解，增量存儲 (Delta Channels)。

底層 Data Layer - Dify

會議記錄，Confluence，Figma 匯入
-> 自動切片 (Chunking) & 向量化。

NotebookLM

實戰演練：會議結束後的自動化工作流



Step 1:
會議逐字稿匯入

僅消耗 1 次 Token

Step 2:
Dify 處理

RAG 入庫 &
產生全局摘要

Step 3:
觸發 LangGraph

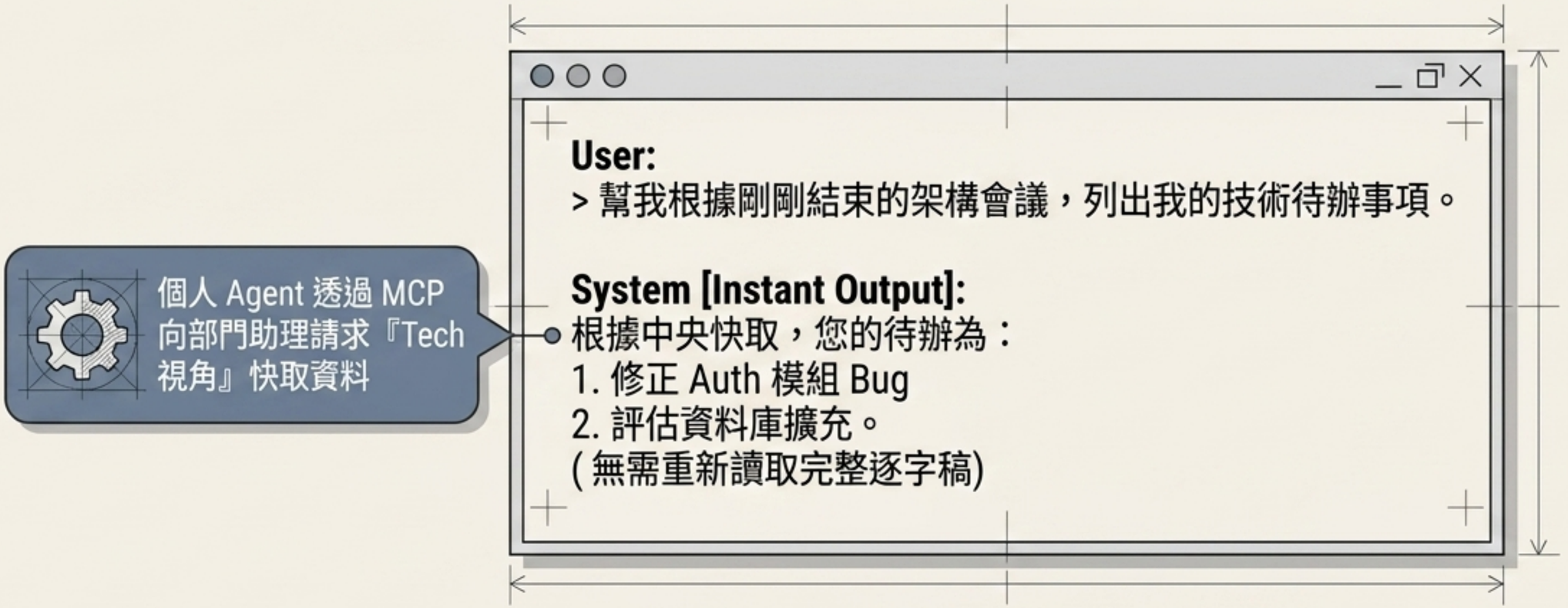
並行節點 (Parallel Edges) 啟動

Step 5:
狀態持久化存檔

Action Items
寫入追蹤系統

NotebookLM

終端體驗：無縫的 Agent-to-Agent 協作



無縫的 Agent-to-Agent 協作，開發者完全不需離開工作環境。

NotebookLM

商業價值與投資回報



成本極致優化

利用中央快取與增量存儲 (Delta Channels) 取代重複調用，API Token 成本指數級下降 (降低 40 倍以上存儲開銷)。

效率最大化

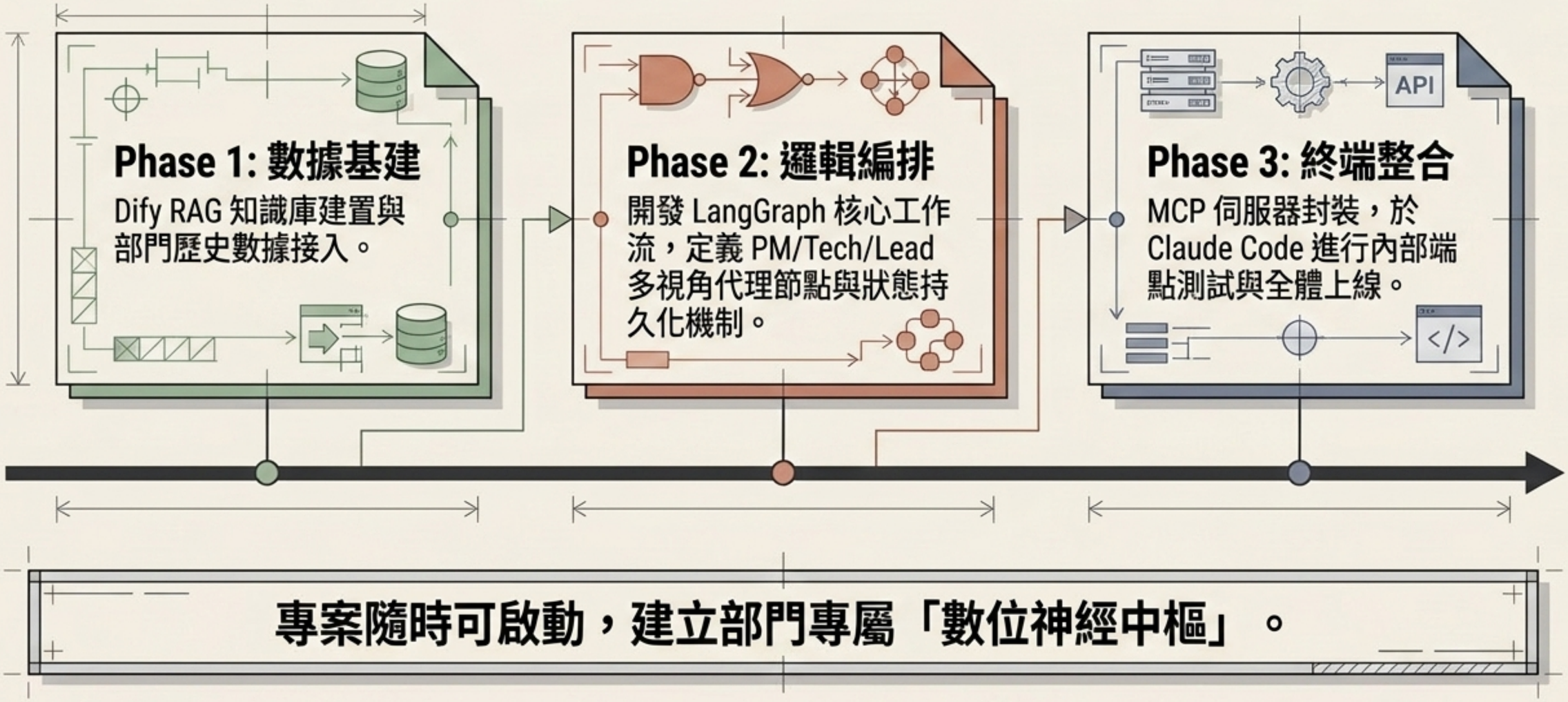
開發者不離開 Claude Code 即可掌握部門全局語境，根絕跨平台切換成本。

營運確定性

透過狀態持久化 (State Memory)，杜絕會議 Action Items 遺漏，實現長期任務狀態 100% 可追溯。

NotebookLM

實施路徑與下一步



NotebookLM